

De La Salle University – Dasmarinas
In Partial Fulfillment of the Requirements for the Subject
Special Topics in Artificial Intelligence

**MASHHUB: AN INTELLIGENT MUSIC LIBRARY MANAGEMENT SYSTEM WITH
FUZZY LOGIC-BASED SONG COMPATIBILITY MATCHING**

A Research Paper Presented to
Ms. Marnelli B. Erquita

By:
BROWN, Yuri M. (Backend Developer)
QUITANEG, Blake Daniel M. (Frontend Developer)
ROSALINAS, Aaron Christian M. (Documentation)
TAN, Adlei Jed D. (AI Model Developer)

Program Code:
BCS42

Date Submitted: May 02, 2026

TABLE OF CONTENTS

ABSTRACT	4
I. INTRODUCTION	5
II. PURPOSE AND DESCRIPTION	13
III. STATEMENT OF THE PROBLEM	15
IV. RESEARCH OBJECTIVES	16
A. General Objective	16
B. Specific Objectives	16
V. PROPOSED SOLUTIONS	17
VI. METHODOLOGY	20
A. Fuzzification	22
B. Rule Base	24
C. Inference Engine	26
D. Defuzzification	28
VII. SYSTEM DESIGN AND ARCHITECTURE	29
A. Presentation Layer	31
B. Domain Layer	32
C. Matching Layer	32
D. Data Access Layer	35

E. Persistence Layer	36
VIII. AI MODEL DESCRIPTION	37
A. Fuzzification: Beats-Per-Minute	38
B. Fuzzification: Musical Key	40
C. Fuzzification: Section	41
D. Rule Base and Inference Engine	42
E. Defuzzification and Ranking.....	44
IX. IMPLEMENTATION	45
A. User Interface	47
B. Fuzzy Logic	53
X. CONCLUSION AND FUTURE ENHANCEMENTS.....	68
REFERENCES	72

ABSTRACT

This paper presents the design, development, and evaluation of MashHub, a music library management and intelligent song compatibility matching system built for disc jockeys and mashup creators. The system addresses the practical challenge of identifying harmonically and rhythmically compatible songs from a curated music database by implementing a Sugeno-type fuzzy logic inference engine at the core of its matching service. Unlike conventional music filtering tools that rely on strict exact-match criteria, MashHub employs fuzzification of beats-per-minute and musical key inputs, a weighted rule-based inference mechanism, and a defuzzification process that ranks candidate songs by aggregate compatibility scores. The system architecture incorporates a section-based song data model that stores tempo and key values at the individual section level, enabling fine-grained compatibility analysis that transcends whole-song averaging. A dual-layer intelligent system combines Fuse.js-powered fuzzy text search with a custom fuzzy inference pipeline for harmonic matching, and a resilient data layer provides cloud-backed storage via Supabase with automatic fallback to local IndexedDB when connectivity is unavailable. The paper discusses the design rationale, algorithmic implementation, and the theoretical basis for applying fuzzy logic to music compatibility, concluding that the approach delivers more musically meaningful and contextually relevant matches compared to rigid filtering methodologies.

Keywords: *fuzzy logic, music information retrieval, BPM matching, harmonic matching, intelligent systems, music library management, mashup creation*

I. INTRODUCTION

The intersection of music technology and artificial intelligence has produced a range of systems designed to assist musicians, producers, and disc jockeys in the discovery, organization, and creative application of music. Among the most persistent challenges in this domain is the problem of song compatibility assessment, which involves determining whether two or more songs can be meaningfully combined, transitioned, or sequenced based on shared or harmonically related musical properties such as tempo and key.

A song is a structured musical composition typically organized into discrete structural units that serve distinct expressive and functional roles within the overall work. These structural units, commonly referred to as sections, include the introduction, verses, pre-choruses, choruses, post-choruses, and bridges, each of which may differ in melodic content, harmonic character, and rhythmic density (Covach, 2005). Beyond its internal structure, a song can be described in terms of two primary musical parameters: its key, which defines the tonal center and the set of pitches around which the composition is organized, and its tempo, measured in beats per minute, which quantifies the rate at which the underlying pulse of the music progresses (Levitin, 2006). These parameters are foundational to the practice of song compatibility assessment, as they determine whether two songs can be combined or transitioned without producing harmonic or rhythmic dissonance.

A mashup is a form of musical composition in which elements from two or more pre-existing recordings are combined to produce a new work, typically by layering the vocal track of one song over the instrumental track of another, or by interleaving segments from multiple

sources into a continuous arrangement (Brøvig-Hanssen and Harkins, 2012). The individual who creates such a composition is commonly referred to as a mashup artist. The practice of mashup creation encompasses a range of distinct formats. The most common form is the two-song mashup, in which a single song is blended with another, often pairing a vocal performance from one recording with an instrumental accompaniment from a second. A song megamix extends this concept by combining the instrumental track of one song with vocal or melodic material drawn from a variety of other sources, producing a medley-style arrangement. Seasonal mashups compile songs associated with a particular time of year, such as a summer mix drawing from recordings released during the first half of a given year, while year-end mashups aggregate the most culturally prominent songs from across a single calendar year into a cohesive retrospective, as exemplified by the widely circulated annual mashup releases that have become a recognized genre of online music production (Gunkel, 2008). Decade mashups take a longer historical view, compiling representative or popular songs from an entire ten-year period, such as a mashup spanning the years 2010 through 2019, to document the sonic character of a musical era.

Disc jockeys occupy a distinct but related creative role in the landscape of music combination and sequencing. In a DJ set, the practitioner curates and transitions between recordings in real time before a live audience, with the goal of maintaining continuous energy, narrative arc, and dancefloor engagement across an extended performance (Brewster and Broughton, 2006). Central to this practice is the technique of transitioning between tracks in ways that preserve musical coherence. One common approach involves aligning the lyrical or vocal phrasing of an outgoing track with the beginning of an incoming track, exploiting natural pauses or phrase boundaries to create perceptually smooth handoffs. However, the feasibility of such

transitions depends critically on the harmonic and rhythmic compatibility of the two tracks. DJs therefore prioritize songs whose tempos are sufficiently close to allow beat matching, either through manual adjustment or digital time-stretching, and whose keys are harmonically adjacent, such as songs that share the same key or occupy neighboring positions on the circle of fifths (Zak, 2001). The practice of harmonic mixing, in which DJs sequence tracks according to key relationships, has been systematized through tools such as the Camelot Wheel, which maps keys to positions that facilitate compatible transitions (Butler, 2006). These considerations collectively establish that tempo and key proximity are not merely desirable properties in a song matching system but are operationally necessary constraints for professional music performance and composition.

A minority of practitioners possess the perceptual ability known as absolute or perfect pitch, which enables them to identify the notes and tonal center of a piece of music through listening alone, without reference to an external standard. For this subset of mashup artists and DJs, key identification is an intrinsic cognitive skill that requires no external tooling. However, the majority of practitioners do not possess this ability and must rely on external resources, whether in the form of curated metadata databases or knowledgeable members of their communities, to obtain accurate harmonic and rhythmic information about the songs they intend to combine.

Online music metadata platforms such as Tunebat and Musicstax represent one such external resource. These services aggregate song-level metadata curated from Spotify's catalog, providing users with artist names, song titles, musical keys, tempo values, and related acoustic attributes through a searchable interface. However, their utility is constrained by several

significant limitations. The coverage of these platforms is effectively bounded by Spotify's own catalog, which is notably incomplete with respect to songs released in Japan. A substantial portion of the Japanese popular music repertoire, including many commercially released J-Pop and anime songs, is absent from Spotify and therefore carries no key or BPM record in these derived databases. Beyond coverage gaps, the accuracy of the available data presents a further problem. A considerable number of entries contain incorrect key or tempo analyses, leading practitioners who rely on this information to produce mashups in which musical elements are harmonically misaligned. This phenomenon, colloquially referred to as an off-key mashup, can arise both from outright analytical errors and from the structural limitation that these databases represent each song with a single global key and a single global BPM value. Songs that undergo key changes between structural sections, for example moving from B Major in the verses to E Major in the choruses, are reduced to a single entry that captures only one of those tonal centers, causing practitioners who consult only this record to remain unaware of the modulation and to make arrangement decisions accordingly.

A third approach, common among more experienced practitioners, involves participation in specialized online communities hosted on the Discord platform, where members collaboratively share and verify harmonic and rhythmic analyses. Western mashup communities such as TheMashupKingdom and MashWorld maintain dedicated channels for the exchange of key and BPM information pertaining to western popular music, and TheMashupKingdom additionally hosts a community-maintained spreadsheet database distributed through Google Sheets. Parallel communities exist for other repertoire niches: K-Pop-focused servers including Let's Mash It Up and MIXXVERSE serve practitioners working with Korean popular music, while

the CosmicMashups Discord server functions as the primary communal resource for anime and J-Pop mashup artists. The analyses shared within these communities are produced by individuals with direct practical experience working on the songs in question, lending them a degree of reliability that curated streaming platform metadata frequently lacks. Furthermore, community-sourced analyses often provide a level of harmonic detail that exceeds what any automated database currently offers. Members routinely document borrowed chords and secondary tonal regions, specify the precise timestamps at which key or tempo changes occur within a song, and identify the modal character of a piece beyond the binary major and minor classification, including modes such as Dorian, Phrygian, Lydian, Mixolydian, Aeolian, and Locrian, each of which carries a corresponding relative major or minor key relationship. Despite these analytical advantages, community-based information retrieval imposes a significant practical burden on the practitioner. Relevant analyses must be located manually by searching through channel histories, and information for recently released or less prominent songs may not yet exist within the community's shared knowledge. The accessibility and timeliness of this resource are therefore inherently dependent on the prior activity of other community members rather than on any systematic or automated process.

Traditional software tools such as Mixed In Key and VirtualDJ are among the most widely adopted within DJ and mashup communities. Unlike online metadata platforms that derive their values from a centralized streaming catalog, these applications operate by allowing users to drag audio files directly into the software interface, whereupon proprietary algorithms scan the raw waveform data to estimate the tempo and tonal center of each track and write the resulting values to the file's ID3 metadata tags. Mixed In Key, which markets itself as a specialist key

detection tool, applies pitch-class profile analysis to the audio signal and outputs results in Camelot wheel notation for use in harmonic mixing workflows. VirtualDJ incorporates analogous BPM and key scanning functionality as part of its broader DJ performance suite, analyzing tracks upon import and storing results in a local database for retrieval during playback preparation. Despite the apparent convenience of these audio-first approaches, their reliability has been demonstrated to fall short of the precision that harmonic mixing and mashup creation demand. A comparative analysis of over two hundred tracks scanned across Mixed In Key, Rekordbox, and Serato DJ found that only thirty-nine percent of the key results were consistent across all three platforms, with Serato and Rekordbox differing from Mixed In Key in forty-five and thirty-eight percent of cases respectively, indicating that the choice of tool alone substantially determines which key a song is assigned rather than reflecting an objective ground truth (Henderson, 2013). Community-based testing of VirtualDJ's key detection on a sample of forty-four songs found a correct identification rate of only approximately fifty percent, with users and developers alike acknowledging on the platform's own support forums that no software currently achieves reliable detection above a two-thirds accuracy ceiling (VirtualDJ Forums, 2019). These limitations are not incidental but structural. Research in Music Information Retrieval has consistently demonstrated that automatic key estimation algorithms perform best on tonally unambiguous, harmonically simple material and degrade substantially when applied to polyphonic recordings with complex instrumentation, modal harmonies, or key changes (Korzeniowski and Widmer, 2017; Wróblewski et al., 2022). Both Mixed In Key and VirtualDJ, along with comparable tools such as Rekordbox and Serato, have been identified as detecting only the standard major and minor modes and failing to recognize songs written in modal scales such as Dorian, Phrygian, or

Lydian, a limitation that misrepresents the tonal character of a substantial portion of contemporary popular and film music (Distinct Mastering, 2024). BPM detection presents parallel challenges: both tools are susceptible to the octave equivalence ambiguity in which a song's tempo is reported at half or double its perceptually correct value, a problem that is particularly prevalent in genres with unconventional rhythmic structures such as trap, drum and bass, and music recorded with live drummers prior to the widespread adoption of click tracks (VirtualDJ Forums, 2022). Beyond their detection limitations, both platforms share with online databases the same fundamental architectural constraint of assigning a single global key and a single global tempo value to each song, precluding any representation of the within-song harmonic and rhythmic variation that is essential information for practitioners constructing section-level mashups.

Fuzzy logic, first formalized by Lotfi A. Zadeh in 1965, offers a principled mathematical framework for reasoning under uncertainty and imprecision. Rather than enforcing binary membership in a set, fuzzy logic assigns degrees of membership, enabling systems to represent the gradated compatibility between musical entities. A song with a tempo of 122 beats per minute is not simply incompatible with a target of 120 beats per minute; it is partially compatible, and the degree of that compatibility can be quantified and incorporated into a meaningful ranking. This capacity for approximate reasoning makes fuzzy logic a theoretically well-suited foundation for music matching systems, where the boundary between compatible and incompatible is inherently continuous rather than discrete.

MashHub was developed to bridge the gap between rigid algorithmic filtering and the nuanced musical judgment of human practitioners. Built with React and TypeScript on the frontend and backed by a resilient dual-backend storage system comprising Supabase for cloud-based persistence and IndexedDB via Dexie for local fallback operation, MashHub provides a comprehensive music library management environment with song metadata storage, project organization, and an advanced matching engine that incorporates fuzzy logic across multiple dimensions of song compatibility. The system integrates all four classical components of a fuzzy inference system: fuzzification, a rule base, an inference engine, and defuzzification. By applying this framework to both beats-per-minute and key compatibility, as well as to section-level musical properties, MashHub produces ranked match results that are more musically coherent and creatively useful than those generated by threshold-based filters.

A distinguishing architectural decision in MashHub is its treatment of songs as multi-section entities rather than as single-valued records. In practice, many songs, particularly in the anime, game, and electronic genres that constitute the primary use cases of the target user community, contain multiple distinct sections with different tempos or keys. A verse may be written in a minor key while the chorus transitions to the relative major; an instrumental introduction may proceed at a different tempo than the main vocal sections. Existing DJ software and library management tools that store only a single global beats-per-minute value and a single key per song cannot represent this internal variation and therefore cannot support compatibility evaluation at the structural level that mashup creation demands. MashHub addresses this gap through its section-based data model and hierarchical fuzzy matching algorithms.

The development of MashHub was driven by the practical needs of disc jockeys and mashup creators who work with large libraries of songs and must rapidly identify candidates for blending, transitioning, or layering. In live performance settings, practitioners assess compatibility across multiple dimensions simultaneously. Tempo determines whether two tracks can be synchronized without rhythmic dissonance. Musical key governs harmonic compatibility; a transition between songs in distantly related keys typically produces an unpleasant clash, while adjacent keys on the circle of fifths blend naturally. Beyond these two primary axes, practitioners also consider vocal arrangement, genre, origin, and the internal section structure of each song. These needs collectively motivate the design choices made throughout the system, including the adoption of fuzzy logic as the theoretical framework for compatibility assessment and the decision to construct a section-first data model that captures intra-song variation at the level of resolution required for mashup planning.

II. PURPOSE AND DESCRIPTION

MashHub is a browser-based music library management application designed to serve as an intelligent workflow companion for DJs and mashup creators. The system is built on a modern web technology stack comprising React 19 and TypeScript for the front-end interface, Dexie 4 as an IndexedDB wrapper for local data persistence, Fuse.js 7 for fuzzy text search, and the dnd-kit library for drag-and-drop interaction. All data is stored locally in the browser, ensuring offline functionality and eliminating dependency on external servers. The architecture is entirely client-side, which aligns with the privacy needs of practitioners who maintain proprietary music libraries.

At the heart of the system is a section-based song data model that separates song metadata from musical properties. Each song record contains standard metadata fields including a unique identifier, title, artist name, type category, origin, release year, season identifier, and optional notes. Musical properties such as tempo and key are stored not at the song level but at the section level, enabling representation of the internal structure of songs that contain multiple distinct sections with differing tempos or keys. This two-tier model, consisting of a songs table and a song sections table with a one-to-many relationship, allows the system to support songs with a single uniform tempo as well as songs that modulate across their duration, which is common in pop, anime, and electronic genres.

Above the data layer sits the matching and search infrastructure, which comprises two distinct systems. The first is a Fuse.js-powered fuzzy text search service that enables typo-tolerant, multi-field search across song titles, artist names, type categories, origin fields, and section part names. This service is configured with a threshold of 0.6, meaning that results must be at least sixty percent similar to the query to appear in results, with weighted field scoring that privileges title and artist matches over type and part matches. The second system is the matching service, which implements a Sugeno-type fuzzy inference system to evaluate song compatibility across BPM, key, title, artist name, and section-specific properties.

The user interface presents these capabilities through a responsive, theme-capable interface with dark and light mode support. A hero section introduces the application with feature badges and call-to-action buttons. A song list view displays all library entries with quick access to detailed modals. A project management system allows users to create named projects,

organize songs into labeled sections within each project, and reorder songs via drag-and-drop. An advanced filters dialog provides access to the full matching interface, including a Quick Match panel that accepts a target song and returns ranked compatibility results, as well as part-specific filter blocks that enable fine-grained BPM and key filtering at the section level.

III. STATEMENT OF THE PROBLEM

Music library management software has historically provided filtering capabilities that operate on the assumption of exact or range-bounded matching. In such systems, a user seeking songs compatible with a target tempo of 120 beats per minute must define an explicit range, for example between 115 and 125 beats per minute, and the system returns all songs whose stored BPM value falls within that range with equal rank. This approach, while operationally simple, introduces several significant limitations that reduce its usefulness for creative practitioners.

1. How can the binary nature of traditional BPM range filtering be transformed into a graded compatibility model that reflects the continuous and perceptual nature of musical tempo similarity?
2. In what way can fuzzy membership functions be used to assign degrees of compatibility between a target tempo and candidate songs, rather than treating all in-range results as equally suitable?
3. How can a multi-dimensional ranking mechanism be developed to aggregate compatibility across tempo, key, and other metadata attributes into a single interpretable match score?

4. How can a fuzzy inference structure be designed to ensure that songs are ranked according to their overall degree of similarity across multiple musical dimensions rather than being evaluated independently?
5. How can section-level musical attributes be incorporated into a compatibility evaluation framework to prevent the loss of potentially valuable matches that are obscured by whole-song averaging?

IV. RESEARCH OBJECTIVES

A. General Objective

The general objective of this research is to design and develop MashHub as an intelligent music library management system that applies Fuzzy Logic principles to the problem of song compatibility matching, demonstrating that a fuzzy inference approach produces more musically meaningful, contextually relevant, and practically useful results than conventional exact-match or range-filter methods for DJs and mashup creators working with large and diverse music libraries.

B. Specific Objectives

- To design and implement a normalized, section-based song data model that stores BPM and key values at the section level rather than at the song level, enabling representation of internally varying musical structures and supporting section-specific compatibility evaluation.

- To implement a Fuse.js-powered fuzzy text search service with similarity thresholds, weighted multi-field scoring, and real-time as-you-type search functionality to support discovery within large music libraries.
- To construct a Sugeno-type fuzzy inference engine that fuzzifies BPM and musical key inputs using membership functions derived from perceptual tolerance ranges, applies a weighted rule base across multiple compatibility dimensions, aggregates rule outputs using weighted additive inference, and defuzzifies the result into a ranked list of candidate songs with explanatory match reasons.
- To extend the fuzzy matching framework to operate at the section level by implementing section name normalization, section-level BPM and key compatibility scoring, and a hierarchical aggregation strategy that combines section-level scores into song-level compatibility ratings.
- To design and implement a project management system that enables users to organize compatible songs into named projects with labeled sections, supporting drag-and-drop reordering.

V. PROPOSED SOLUTIONS

MashHub is designed as a browser-based, client-side application that requires no server-side computation for its core matching and search functionality. The system provides a unified environment for music library management, project organization, and intelligent compatibility matching, with all user data stored either in a cloud-backed Supabase database or locally in the

browser via IndexedDB, depending on connectivity and configuration. This architectural choice aligns with the privacy needs of practitioners who maintain proprietary music libraries and reflects the practical requirement for offline operation during live performance settings where network connectivity may be unreliable.

The section-based song data model is the structural foundation of the system and distinguishes MashHub from conventional music library tools. Each song record contains standard metadata fields including a unique identifier, title, artist name, type category, origin, release year, season identifier, and optional notes. Musical properties such as tempo and key are stored not at the song level but at the section level through a one-to-many relationship between songs and song sections. Each section record contains a section identifier, a reference to the parent song, a part label such as Intro, Verse, Chorus, or Bridge, a beats-per-minute value, a musical key, and a section order index that specifies the sequential position of the section within the song. This two-tier model allows the system to represent songs with a single uniform tempo as well as songs that modulate across their duration, supporting both simple and complex harmonic structures within a single unified data schema.

The project management system provides the primary organizational layer through which practitioners structure their creative outputs. Projects are named containers with a type designation of Seasonal, Year-End, Song Megamix, or Other, and each project type controls how the project is visualized in the interface. Projects are subdivided into first-class project sections, each of which has a name, an order index, and optional harmonic constraints including a target beats-per-minute value, a beats-per-minute range, a target key, and a Camelot key range. These

section-level constraints are used in the suggestion drawer to surface compatible candidates for addition to a specific section, providing context-aware recommendations that reflect the harmonic and rhythmic requirements of the specific part of the set being planned. Songs are added to projects through project entries that reference both a project section and a song, with additional fields for ordering, locking, and per-entry notes.

The advanced search and matching infrastructure addresses two distinct user tasks through separate but coordinated systems. The first is the Fuse.js-powered fuzzy text search service, which enables typo-tolerant, multi-field search across song titles, artist names, type categories, origin fields, and section part names. This service is configured with a similarity threshold of 0.6, a maximum character distance of 100, and weighted field scoring that assigns forty percent of the search relevance to title matches, thirty percent to artist matches, fifteen percent to type matches, ten percent to origin matches, and five percent to section part matches. The index is built once on application initialization and updated incrementally as songs are added, modified, or removed, ensuring that search results remain current without requiring full index reconstruction on each query. Real-time search feedback is provided through debounced input processing with a three-hundred-millisecond delay that prevents excessive computation during rapid typing.

The second and algorithmically more sophisticated system is the fuzzy matching service, which evaluates song compatibility across beats-per-minute, key, vocal status, type, release year, text content, and section-specific properties. This service implements two primary matching pathways: a standard matching pathway that applies a weighted rule base across all active filter

criteria to produce a global compatibility score for each candidate, and a quick matching pathway that focuses specifically on section-level harmonic and rhythmic compatibility between a selected target song and all other songs in the library. Both pathways produce ranked result lists annotated with component scores and linguistic match reasons, enabling users to inspect the specific basis for each recommendation.

The filtering system provides access to the matching infrastructure through both inline filter controls and an advanced filters dialog. Inline filters expose primary harmonic criteria including beats-per-minute targeting, key selection, and year range directly below the search bar for rapid access. The advanced filters dialog provides access to the full matching interface including the Quick Match panel, part-specific filter blocks that enable beats-per-minute and key constraints at specific section levels, and additional metadata filters for type, origin, season, and artist. Part-specific filter blocks can be combined in AND logic to express complex structural constraints such as finding songs whose verse sections are in a particular key and whose chorus sections fall within a specific tempo range, a capability with no equivalent in existing music library management tools.

VI. METHODOLOGY

The matching engine of MashHub implements a Sugeno-type fuzzy inference system, a variant of fuzzy logic in which rule consequents are crisp constants or linear functions rather than fuzzy sets, enabling efficient defuzzification through weighted averaging. The system operates across four stages: fuzzification of crisp inputs, evaluation of a weighted rule base, aggregation of rule outputs through weighted summation, and defuzzification by sorting and ranking. Each

stage is described in detail below, with reference to the specific implementation artifacts in the codebase.

The dataset underlying MashHub was assembled through a combination of personal curation and collaborative contribution from established communities within the mashup artist ecosystem. The initial database was developed by Yuri Brown, one of the researchers and a 10-year experienced mashup artist specializing in anime song mashups, through the direct analysis and curation of Japanese-language music as part of his professional creative work. This initial dataset comprised songs drawn primarily from the J-Pop genre as well as opening and ending theme songs from Japanese anime series, each annotated with metadata including song title, artist, tempo, key, and section-level musical properties derived from Brown's own analytical work. The scope of the dataset was subsequently expanded through collaboration with contributors from two external communities within the online mashup artist network.

The first expansion was facilitated by MixMstrStel, a moderator of TheMashupKingdom, a western mashup community operating through the Discord platform and commonly referred to by its members as TMK. Through this collaboration, MashHub was granted access to the server's curated database of more than 3,000 western songs, the analyses of which were contributed collectively by the mashup artists affiliated with that community. The second expansion was made possible through the assistance of PRDSM, a mashup artist who provided access to the analytical database of more than 3,000 K-Pop songs maintained by an initial Discord bot within the server known as Let's Mash It Up, or LMIU, which serves as a community resource for sharing song keys and beats-per-minute analyses contributed by its member artists. The

integration of these three datasets produced a unified corpus of nearly 8,000 songs spanning multiple song types, including Anime, Western, and K-Pop, and encompassing a wide range of musical genres. Because the constituent datasets originate from independent communities employing their own analytical methods and conventions, the aggregated database reflects the diverse perspectives and practical expertise of multiple mashup practitioners rather than a single standardized analytical framework.

A. Fuzzification

Fuzzification is the process by which crisp numerical and categorical inputs are transformed into graded membership values ranging continuously from zero to one, representing the degree to which an input satisfies a given fuzzy set. MashHub implements three distinct fuzzification procedures corresponding to BPM proximity, key compatibility, and section name normalization.

BPM fuzzification is implemented in the `bpmMatching.ts` utility module through two functions that apply different membership curve shapes depending on the operational mode. The standard matching function, `getBpmCompatibilityScore`, accepts an array of BPM values from a candidate song's sections and a target BPM derived from the active filter criteria. It identifies the section BPM in the candidate array that minimizes the absolute distance from the target and applies a linear decay formula to produce a membership value. If the minimum distance exceeds the caller-specified maximum delta, a membership of zero is returned immediately. Otherwise, the membership is computed as the maximum of zero and the quantity one minus the ratio of the best distance to the constant `BPM_SCORE_DENOMINATOR`, which is

set to 22.5. This produces a trapezoidal-style membership function within its active domain, in which BPM differences of zero yield full membership and differences approaching 22.5 BPM yield near-zero membership.

A secondary fuzzification pathway is provided by the `areBpmsHarmonicallyRelated` function, which checks whether the ratio of two BPM values approximates one of the standard harmonic ratios: 2.0, 3.0, 4.0, 1.5, 0.5, 0.75, or 1.33. A configurable ratio tolerance defines the width of the fuzzy membership around each ratio, allowing songs at double or half tempo to be identified as harmonically related even when they do not match the target BPM directly. In the quick matching service, a simplified membership function applies the formula one minus the BPM difference divided by 22.5, with a floor of zero, to produce a compatibility score for section-level BPM comparisons.

Key fuzzification is implemented in the `calculateKeyDistance` function within the `keyNormalization` utility module. Musical keys are represented as pitch-class and mode pairs, where the pitch class is an integer from zero to eleven representing the twelve chromatic pitches and the mode is a binary indicator for major or minor tonality. The fuzzification proceeds through a three-tier evaluation. When the candidate key and the target key share the same pitch class and the same mode, a membership degree of 1.0 is assigned, representing exact compatibility. For all other cases, the circular semitone distance between the two pitch classes is computed using the formula for minimum arc length on a twelve-element circular structure: the minimum of the absolute difference and twelve minus the absolute difference. This circular distance ranges from zero for identical pitch classes to six for the tritone, the most dissonant interval in Western

harmony. An exact pitch-class match yields a membership of 1.0 regardless of modality. For non-zero distances, the function follows a bell-shaped decay curve in which one semitone distance maps to approximately 0.95, two semitones to 0.90, three semitones to 0.80, and six semitones to 0.0. This curve provides greater tolerance for harmonically adjacent keys than a purely linear function would, reflecting the perceptual reality that keys one or two semitones apart retain substantial tonal overlap in practice. The use of a pre-computed 12 by 12 lookup table, designated the SEMITONE_DISTANCE_MAP, enables fast retrieval of circular distances without repeated arithmetic computation.

Section name fuzzification is implemented in the `normalizeSectionName` function within the `sectionNormalization` utility. This function maps variable and extended section names, such as "Verse A", "Verse B" and "Verse 2", to a set of canonical base section names including Intro, Verse, Prechorus, Chorus, Bridge, and Other. The mapping is performed by checking whether the input section name begins with or contains any of the canonical labels. This process transforms the discrete space of section names into a fuzzy set of base categories, enabling the matching system to treat "Verse A" and "Verse B" as members of the same fuzzy section class with degree 1.0, while treating "Verse" and "Chorus" as members of distinct classes. Section normalization underlies the section-level compatibility evaluation described in the inference section below.

B. Rule Base

The rule base of a fuzzy inference system consists of IF-THEN rules that relate linguistic input conditions to weighted output contributions. In MashHub, the rule base is encoded as weighted scoring conditions within the `evaluateMatch` and `getQuickMatches` functions of the

matching service. All rule weights are defined as named constants in the system constants module, enabling systematic adjustment without modification of the inference logic.

For the standard song matching pathway implemented in `evaluateMatch`, the rule base comprises five primary rules. The first rule states that if the candidate BPM is close to the target, the match score contribution is the product of the BPM compatibility membership degree and the BPM weight of 0.45. The second rule states that if the candidate key falls within the selected key set, or within a target key plus tolerance, the match score contribution is the product of the key compatibility membership degree and the key weight of 0.45. The third rule states that if the candidate song title matches the filter title, a fixed bonus equal to the title weight of 0.05 is added. The fourth rule states that if the candidate artist name matches the filter artist name, a fixed bonus equal to the artist weight of 0.05 is added. The aggregate weight of all four rules is 1.0, representing perfect compatibility across all evaluated dimensions.

For the quick matching pathway implemented in `getQuickMatches`, which evaluates section-specific compatibility between a target song and candidate songs, the rule base is reformulated to emphasize section-level properties. The first rule in this pathway states that if the part-specific keys of matching sections are harmonically similar, the contribution is the product of the section-level key compatibility score and the quick match key weight of 0.45. The second rule states that if the part-specific BPMs of matching sections are harmonically related, the contribution is the product of the section-level BPM compatibility score and the quick match BPM weight of 0.45. A third rule adds a bonus of 0.05 when the candidate and target share the same artist, and a fourth rule adds 0.05 when they share similar song titles. The equal weighting

of key and BPM in the quick matching pathway reflects the equal importance of tonal and rhythmic compatibility in the context of section-level mashup planning.

C. Inference Engine

The inference engine applies the rules to the fuzzified inputs and computes the aggregate output. MashHub implements a weighted additive aggregation strategy consistent with the Sugeno-type inference model, in which each rule that fires contributes a product of its firing strength and its weight to the total match score. Rules that do not fire, because the relevant filter criterion was not specified or the membership degree is zero, contribute nothing to the aggregate.

At the section level, the inference is structured as a two-stage hierarchical process implemented in the `calculatePartSpecificKeyScore` function. In the first stage, for each section in the target song, the inference engine identifies all corresponding sections in the candidate song by matching normalized section names. When matching candidate sections exist, the key compatibility is evaluated using the `calculateKeyDistance` fuzzification function. If either the target or candidate section contains multiple key values, all pairwise distances between target and candidate keys are computed, and the mean of all pairwise compatibility scores is retained. This mean aggregation strategy reflects the musical reasoning that the overall harmonic relationship between two songs at a given section should account for all key pairings present, not just the most favorable one — producing a score that is representative of the section's typical compatibility rather than its best-case alignment. In the second stage, the section-level scores produced by the first stage are averaged across all sections for which a match was found. This

mean aggregation ensures that the final section-level score reflects overall section compatibility rather than being dominated by a single well-matched section. The result of this two-stage inference, a value in the range zero to one, is then passed to the top-level scoring function where it is weighted according to the quick match key rule.

A parallel process applies to BPM compatibility at the section level. For each pair of matching sections, the BPM compatibility is evaluated using the simplified linear decay formula, and the resulting scores are aggregated across sections to produce a single section-level BPM compatibility rating. When a candidate song does not have section data but has a whole-song key or BPM value, a fallback mechanism compares the whole-song value against each section in the target song, providing partial compatibility scoring even when section data is unavailable for the candidate. This full-song fallback ensures that the system can still evaluate candidate songs whose section information has not been entered, at the cost of reduced precision.

For the harmonic matching pathway implemented in `findHarmonicMatches`, the inference engine applies a simplified rule set that does not use continuous membership functions but instead fires discrete rules based on threshold conditions. A score of 0.5 is added when the BPM matching function identifies a direct or harmonically related tempo relationship, 0.3 when the key matching function identifies a compatible key, and 0.1 when the song type matches the target. While this pathway does not implement graduated fuzzification, it operates consistently with the broader fuzzy logic framework by using additive rule aggregation to rank candidates by multi-dimensional compatibility.

D. Defuzzification

Defuzzification is the final stage of the fuzzy inference process, in which the aggregated fuzzy output is converted into a crisp decision or action. In MashHub, the aggregate match score computed by the inference engine is already a scalar value in the range zero to one, a characteristic of Sugeno-type inference systems where defuzzification reduces to the computation of the weighted average itself. The crisp decision produced by the system is the ranking of candidate songs by descending match score, implemented through a standard sort operation on the result list.

The sorted result list constitutes the defuzzified output: the song with the highest aggregate score is the first recommendation, followed in order by songs of decreasing compatibility. This ranking is the primary actionable output of the matching system and is presented to the user in the Quick Match panel and filter results views. To support interpretability and transparency, the system additionally produces a set of linguistic match reasons for each result, including degrees such as "Low", "Medium", and "High". These reasons are derived from the component scores and firing conditions of the individual rules, and they enable users to understand why a particular song was ranked highly without requiring familiarity with the underlying fuzzy logic implementation. The combination of crisp ranking and linguistic explanation thus constitutes a complete defuzzification output that is both computationally precise and humanly interpretable.

Affinity categories are additionally derived from the aggregate score to provide a simplified three-level classification: scores above 0.7 are classified as High affinity, scores

between 0.4 and 0.7 as Medium affinity, and scores below 0.4 as Low affinity. These categories are presented as color-coded badges in the Quick Match panel, offering an immediately accessible summary of match quality for users who do not wish to inspect component scores in detail. This categorical output represents a secondary defuzzification step that maps the continuous match score to a discrete quality label appropriate for rapid scanning of result lists.

VII. SYSTEM DESIGN AND ARCHITECTURE

MashHub is structured as a five-layer client-side architecture in which each layer maintains a clearly defined responsibility, enabling separation of concerns across the presentation, domain modeling, fuzzy inference, data access, and persistence concerns of the system. The layers are organized such that data flows vertically from user interaction at the topmost presentation layer down through domain abstractions and a fuzzy matching engine before reaching the persistence backends, with each layer depending only on the abstractions exposed by the layer immediately beneath it.

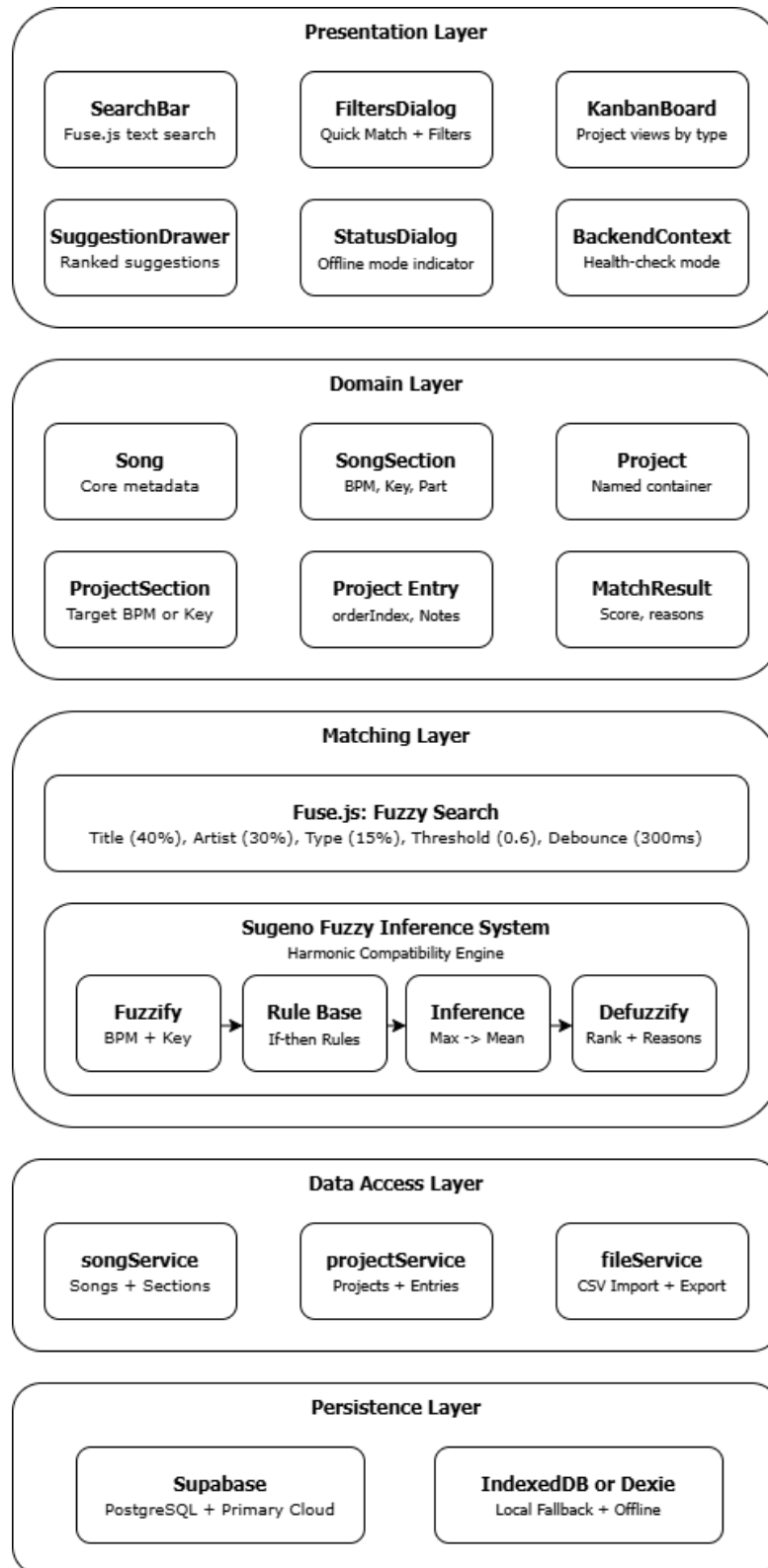


Figure 7.1. System Architecture.

A. Presentation Layer

The presentation layer is implemented using React 19 with TypeScript and constitutes the entirety of MashHub's user-facing interface. It comprises six primary components whose responsibilities are partitioned according to distinct interaction concerns. The `AdvancedSearchBar` component serves as the real-time entry point for text-based queries, delegating input directly to the Fuse.js fuzzy search subsystem with a debounce interval of 300 milliseconds to limit unnecessary index evaluations during rapid keystrokes. The `AdvancedFiltersDialog` component functions as the primary control surface for the harmonic matching pipeline, allowing users to construct part-specific filter criteria and to invoke the Quick Match feature, which triggers the Sugeno-type fuzzy inference system against a selected reference song. The `KanbanBoard` and `MegamixTimeline` components render project workspaces according to the project type, with Kanban views applied to seasonal and year-end project types and the Megamix timeline applied to song megamix projects. The `SuggestionDrawer` component surfaces ranked compatibility recommendations within a project section context, consuming ordered `MatchResult` objects produced by the matching layer. The `ConnectionStatusDialog` component monitors backend availability and presents non-blocking feedback to the user when the system operates in offline mode. Finally, the `BackendContext` component performs the health-check orchestration that determines whether the system routes data operations to Supabase or to the local IndexedDB fallback, broadcasting mode transitions to all dependent components.

B. Domain Layer

The domain layer defines the TypeScript type system that governs data representation throughout the application. The `Song` type encodes core bibliographic metadata including title, artist, type, origin, year, and season, but notably excludes direct BPM and key fields, as these properties are delegated to the section level in keeping with the system's section-first data model. The `SongSection` type captures the musical properties of an individual named section within a song, storing the part label, BPM value, musical key, and sequential section order, thereby allowing a single song to exhibit multiple distinct harmonic regions across its structure. The `Project`, `ProjectSection`, and `ProjectEntry` types collectively model the organizational container through which users curate songs into structured, ordered sequences. `ProjectSection` additionally carries optional harmonic constraint fields including target BPM, BPM range bounds, and target key, which the matching layer references when computing compatibility scores within a project context. The `MatchResult` type represents the output of the fuzzy inference pipeline, carrying a scalar `matchScore`, component scores for BPM and key, and an array of human-readable reason strings that expose the scoring rationale to the user interface layer.

C. Matching Layer

The matching layer constitutes the primary intellectual contribution of the MashHub system and is implemented as two structurally independent fuzzy subsystems that operate over the song and section data provided by the domain layer.

The first subsystem is a fuzzy text retrieval engine constructed using Fuse.js version 7, which provides typo-tolerant, multi-field approximate string matching over song metadata. The

search index is configured with a similarity threshold of 0.6, a maximum character distance of 100, and a minimum matching character length of 2. Field weights are assigned in proportion to their expected identificatory value, with the title field receiving the highest weight of 0.40, followed by the artist field at 0.30, the type field at 0.15, the origin field at 0.10, and the part field at 0.05. The index is maintained incrementally, meaning that individual song additions, updates, and removals are reflected without requiring a full index reconstruction. This subsystem is integrated directly into the `AdvancedSearchBar` component and drives real-time search suggestions as the user types.

The second subsystem is a custom Sugeno-type fuzzy inference system designed specifically for harmonic compatibility assessment between songs and song sections. This system implements the four classical stages of fuzzy logic processing: fuzzification, rule evaluation, inference, and defuzzification. During fuzzification, BPM proximity is measured through two distinct membership functions. The standard matching function applies a trapezoidal linear decay with a denominator constant of 22.5, while the Quick Match function applies a piecewise sigmoid-style curve in which BPM differences of zero, five, and ten produce membership values of 1.0, 0.9, and 0.8 respectively, with differences beyond ten decaying linearly to zero at twenty BPM. Key compatibility is computed by the `calculateKeyDistance` function using circular semitone distance over the twelve-tone equal temperament pitch class space. An exact pitch-class match yields a membership of 1.0 regardless of modality. As the circular semitone distance increases from zero toward six, the membership function follows a bell-shaped decay curve rather than a purely linear one, producing reference values of approximately 0.95 at one semitone, 0.90 at two semitones, and 0.80 at three semitones, reaching 0.0 at six semitones corresponding to the

tritone interval. The `getQuickMatchKeyScore` function is an alias of `calculateKeyDistance` and applies the same bell-shaped decay in both standard and Quick Match modes. Importantly, an exact pitch-class match between two keys is awarded a membership of 1.0 regardless of whether the two keys share the same modality, reflecting the practical interchangeability of parallel major and minor keys in harmonic mixing. The Quick Match key scoring function `getQuickMatchKeyScore` is an alias of `calculateKeyDistance` and applies the identical linear decay rule. Section name fuzzification is handled by `normalizeSectionName`, which maps diverse label variants such as "Verse A" and "Verse 2" onto a canonical vocabulary comprising Intro, Verse, Prechorus, Chorus, Bridge, and Other.

The rule base implements weighted IF-THEN conditions with singleton Sugeno consequents. The standard matching mode applies four rules: BPM compatibility weighted at 0.45, key compatibility weighted at 0.45, exact artist correspondence weighted at 0.05, and title text containment weighted at 0.05, for a maximum achievable score of 1.0. The Quick Match mode applies a parallel four-rule set in which part-specific key similarity and part-specific BPM compatibility are each weighted at 0.45, and same-artist and same-origin bonuses are each weighted at 0.05. Candidates whose type differs from the reference song's type are excluded by a hard pre-filter before scoring proceeds.

The inference engine aggregates rule outputs through weighted additive composition. Section-level key scoring is performed by the `calculatePartSpecificKeyScore` function, which implements a two-stage hierarchical aggregation. For each section in the reference song, all matching sections in the candidate song are identified by normalized part name. The key

similarity is then computed for every pairwise combination of keys across the matching sections, and the mean of all pairwise similarities is taken as the score for that target section. This per-section mean is then averaged across all matched section pairs to produce a single part-specific key compatibility score. Section-level BPM scoring follows the same mean pairwise aggregation strategy. When a candidate song contains no section-level data, a full-song key fallback computes the mean similarity between the candidate's global key and each section key of the reference song.

Defuzzification converts the aggregated weighted score into a crisp ranking by sorting all candidate results in descending order of their scalar `matchScore`. Each result is additionally annotated with affinity category labels, where scores above 0.7 are classified as high affinity, scores between 0.4 and 0.7 as medium affinity, and scores below 0.4 as low affinity, alongside an array of linguistic reason strings that communicate the specific harmonic relationships underlying each score.

D. Data Access Layer

The data access layer encapsulates all storage operations behind three service abstractions that insulate the presentation and domain layers from the details of the underlying persistence mechanism. The `songService` manages all create, read, update, and delete operations for songs and their associated sections, routing each operation through the `withFallback` pattern to determine whether execution should proceed against Supabase or against the local `IndexedDB` store. The `projectService` provides equivalent operations for projects, project sections, and project entries, including section reordering, entry note management, and the construction of

fully enriched `ProjectWithSections` objects that aggregate section and song data for rendering in the project management views. The `fileService` handles CSV import and export in a two-file format separating song metadata and section data, and is responsible for seeding the local `IndexedDB` store with initial data on the first offline use of the application.

E. Persistence Layer

The persistence layer implements a dual-backend strategy governed by the `withFallback` higher-order function, which wraps every service method with a try-primary, fall-back-to-local execution pattern. On application startup, `BackendContext` issues a lightweight health-check query to Supabase with a five-second timeout. If the query succeeds, the system operates in Supabase mode and all data operations are directed to the PostgreSQL backend hosted on the Supabase platform, where the relational schema mirrors the domain layer's logical structure across the `songs`, `song_sections`, `projects`, `project_sections`, and `project_entries` tables. If the health check fails, or if a Supabase operation fails at any point during a session, the system transitions to local mode and all subsequent operations are directed to an `IndexedDB` store managed by Dexie version 4. The same logical schema is implemented in Dexie using camelCase field naming, and the service layer performs the necessary field mapping between the two representations transparently. When operating in local mode, the `ConnectionStatusDialog` is rendered to inform the user of the offline state and to provide a retry mechanism that re-executes the health check and, upon success, refreshes all application data from Supabase.

VIII. AI MODEL DESCRIPTION

MashHub implements a Sugeno-type fuzzy inference system for music compatibility matching, a form of computational intelligence that is categorically distinct from machine learning approaches. Machine learning systems, including neural networks and gradient boosting models, acquire their decision-making behavior from training data through iterative parameter optimization and typically produce opaque representations whose internal structure resists direct musical interpretation. The fuzzy inference system in MashHub, by contrast, encodes musical compatibility knowledge explicitly as membership functions and weighted rules, all parameters of which are visible, interpretable, and adjustable by anyone with domain knowledge. This transparency is not merely a philosophical preference but a practical requirement for a creative workflow tool: practitioners must be able to understand, evaluate, and if necessary override the system recommendations based on their own musical judgment.

The fuzzy model operates in two distinct modes with separate rule bases and membership functions. The standard matching mode, implemented in the `evaluateMatch` function of the matching service, applies a broad multi-criteria rule base across all active filter criteria to score any candidate song against a set of specified target properties. The quick matching mode, implemented in the `getQuickMatches` function, focuses specifically on section-level harmonic and rhythmic compatibility between a selected target song and all other songs in the library, using dedicated membership functions with piecewise curves designed to maintain reasonable compatibility scores across the practical tolerance ranges of experienced practitioners. Both modes share the same four-component pipeline of fuzzification, rule base evaluation, inference

aggregation, and defuzzification, but they differ in their specific membership functions, rule structures, and aggregation strategies.

A. Fuzzification: Beats-Per-Minute

Fuzzification is the process of transforming crisp numerical inputs into graduated membership degrees in the range zero to one. For beats-per-minute compatibility in standard matching mode, the `getBpmCompatibilityScore` function accepts a candidate song with potentially multiple beats-per-minute values across its sections and a target beats-per-minute value. The function identifies the candidate beats-per-minute value closest to the target by minimizing absolute difference. If no candidate value exists within a specified maximum delta threshold, the membership degree is zero, indicating complete incompatibility. Otherwise, the membership degree is computed as the maximum of zero and one minus the ratio of the closest matching distance to the denominator constant 22.5. This produces a trapezoidal-style membership function that assigns a degree of 1.0 when the candidate exactly matches the target, decreases linearly as the distance increases, and reaches zero at a distance of 22.5 beats per minute. The denominator value of 22.5 was derived as fifteen multiplied by 1.5 and was chosen to reflect the practical tempo tolerance of experienced practitioners evaluating mixing compatibility.

For quick matching mode, the `getQuickMatchBpmScore` function applies a piecewise sigmoid-style curve calibrated to maintain acceptable compatibility scores across the tolerance range relevant to section-level comparison. Songs with no beats-per-minute difference receive a score of 1.0 representing perfect tempo alignment. Songs five beats per minute apart receive a

score of 0.9, reflecting high but not perfect alignment. Songs ten beats per minute apart receive a score of 0.8, indicating that synchronization requires modest tempo adjustment but remains practical. Beyond ten beats per minute, the score decreases from 0.7 at eleven beats per minute difference to zero at twenty beats per minute, reflecting the point at which tempo differences become perceptually prominent even for practitioners with beatmatching skill. This formula, expressed as one minus the product of the difference and 0.02 for differences up to ten, and as 0.7 minus the product of the excess over eleven and 0.0778 for larger differences, produces a piecewise curve with a moderate decay rate that preserves meaningful differentiation across the practical compatibility range.

A supplementary fuzzification pathway for beats-per-minute is provided by the `areBpmsHarmonicallyRelated` function, which evaluates whether the ratio of two beats-per-minute values approximates one of the standard harmonic ratios: 2.0, 3.0, 4.0, 1.5, 0.5, 0.75, and 1.33. These ratios correspond to double tempo, triple tempo, and their reciprocals, as well as the common half-step and two-thirds relationships used in DJ practice. A configurable tolerance of four percent around each ratio defines the width of the fuzzy membership, allowing songs at nominally double or half tempo to be identified as harmonically related even when their stored values do not exactly satisfy the ratio condition. This pathway is used in the `findHarmonicMatches` function as a fallback when section-level data is unavailable and provides a complementary perspective on tempo compatibility that operates on the harmonic structure of the tempo relationship rather than simple proximity.

B. Fuzzification: Musical Key

Key fuzzification is implemented through two functions that share a common theoretical basis in circular semitone distance but differ in their membership curve shapes. The `calculateKeyDistance` function, used in standard matching mode, represents musical keys as pitch-class and mode pairs, where the pitch class is an integer from zero to eleven encoding the twelve chromatic pitches and the mode is a binary indicator for major or minor tonality. The function applies a three-tier evaluation. When the candidate and target keys share both the same pitch class and the same mode, the membership degree is 1.0, representing exact compatibility. When they share the same pitch class but differ in mode, the membership degree is 0.85, representing high but imperfect compatibility with a fixed penalty of 0.15 for the mode difference. This penalty reflects the musical reality that parallel major and minor keys share the same tonal center and many common harmonies but produce a perceptibly different emotional character that not all transitions can accommodate gracefully.

For all other pitch-class pairs, the function computes the circular semitone distance using the minimum arc length formula on a twelve-element circular structure: the minimum of the absolute difference between pitch classes and twelve minus that difference. This distance ranges from zero for identical pitch classes to six for the tritone, the most dissonant interval in Western tonal harmony. The membership degree is then computed as one minus the ratio of the circular distance to the maximum semitone distance constant of six, producing a linear decay from 1.0 at zero chromatic distance to 0.0 at the tritone. A precomputed twelve-by-twelve lookup table

designated SEMITONE_DISTANCE_MAP provides fast retrieval of circular distances without repeated arithmetic, supporting efficient evaluation across large candidate sets.

The getQuickMatchKeyScore function, used in quick matching mode, applies a piecewise curve analogous to the beats-per-minute quick match function. Keys sharing the same pitch class and mode receive a score of 1.0. Keys one semitone apart receive a score of 0.9, reflecting near-unison compatibility that practitioners routinely exploit for minor-second transitions. Keys two semitones apart receive a score of 0.8. Keys three or more semitones apart receive a score decreasing from 0.7 at three semitones to zero at the tritone distance of six semitones. This calibration ensures that the most common harmonic relationships employed in DJ mixing, those within two semitones on the chromatic scale, receive high compatibility scores while more distant relationships are ranked proportionally lower.

C. Fuzzification: Section

Section names in music libraries exhibit substantial variation in labeling conventions. A given verse section may be labeled as Verse, Verse A, Verse B, Verse 2, or Verse Bridge depending on the annotator. For section-level matching to function correctly across songs annotated by different practitioners or imported from different sources, the system must recognize that these variant labels refer to the same structural category. The normalizeSectionName function in the section normalization utility addresses this by mapping variable and extended section names to a set of six canonical base categories: Intro, Verse, Prechorus, Chorus, Bridge, and Other. The mapping is performed by checking whether the input section name begins with or contains one of the canonical labels, with longer matches taking precedence over shorter ones to prevent

misclassification. This process transforms the discrete space of section labels into a fuzzy set of base categories, enabling the matching system to treat Verse A and Verse B as members of the same fuzzy section class with full membership degree while treating Verse and Chorus as members of distinct classes.

D. Rule Base and Inference Engine

The rule base in standard matching mode comprises five primary weighted rules encoded in the `evaluateMatch` function. The first rule states that if the candidate beats-per-minute is close to the target, the match score contribution is the product of the beats-per-minute compatibility membership degree and the beats-per-minute weight of 0.40. The second rule states that if the candidate key falls within the selected key set or within a target key plus tolerance, the contribution is the product of the key compatibility membership degree and the key weight of 0.30. The third rule states that if the candidate song type matches the filter type, a fixed bonus equal to the type weight of 0.10 is added. The fourth rule states that if the candidate release year falls within the specified year range, a fixed bonus equal to the year weight of 0.05 is added. The fifth rule states that if the candidate song title contains the text search query, a fixed bonus equal to the text weight of 0.05 is added. Additional contributions from vocal status matching can bring the total possible aggregate score to 1.0, representing perfect compatibility across all evaluated dimensions.

The quick matching rule base is reformulated to emphasize section-level properties. The first rule in this pathway states that if the part-specific keys of matching sections are harmonically similar as evaluated by the piecewise key curve, the contribution is the product of the section-

level key compatibility score and the quick match key weight of 0.45. The second rule states that if the part-specific beats-per-minute values of matching sections are closely related as evaluated by the piecewise tempo curve, the contribution is the product of the section-level tempo compatibility score and the quick match beats-per-minute weight of 0.45. A third rule adds a bonus of 0.05 when the candidate and target share the same artist, and a fourth rule adds 0.05 when they share the same origin. The equal weighting of key and beats-per-minute in the quick matching pathway reflects the equal practical importance of tonal and rhythmic compatibility in the context of section-level mashup planning.

The inference engine applies these rules to the fuzzified inputs through weighted additive aggregation consistent with the Sugeno-type model. Each rule that fires contributes a product of its firing strength and its weight to the total match score, and rules that do not fire contribute nothing. At the section level, the inference is structured as a two-stage hierarchical process implemented in the `calculatePartSpecificKeyScore` function. In the first stage, for each section in the target song, the inference engine identifies the corresponding section in the candidate song by matching normalized section names. When a matching candidate section is found, key compatibility is evaluated using the relevant key distance function. If either section contains multiple key values, all pairwise distances between target and candidate keys are computed and the maximum pairwise compatibility score is retained, reflecting the musical convention that two songs are considered key-compatible at a section if at least one key pairing between them is harmonically close. In the second stage, the section-level scores produced by the first stage are averaged across all sections for which a match was found, producing a single song-level section compatibility rating that reflects overall structural harmonic alignment.

A fallback mechanism handles the case where a candidate song lacks section data but has a stored whole-song key or beats-per-minute value. In this situation, the whole-song value is compared against each section of the target song, and the resulting compatibility scores are averaged to produce a partial song-level rating. This approach provides meaningful scoring for candidate songs that have not yet been annotated at the section level, at the cost of reduced precision relative to the full section-level evaluation. The fallback ensures that the system can evaluate all songs in the library regardless of their annotation completeness, which is important for practical usability during the incremental process of building and annotating a large music library.

E. Defuzzification and Ranking

Defuzzification in MashHub converts the aggregated match score produced by the inference engine into a crisp decision appropriate for presentation to the user. Because the Sugeno-type inference model produces a scalar weighted average as its aggregate output, the match score is already a crisp value in the range zero to one, and defuzzification reduces to the computation of that weighted average itself. The primary crisp output of the system is the ranking of candidate songs by descending match score, implemented through a standard sort operation on the result list. The song with the highest aggregate score appears first in the recommendation list, followed in order by songs of decreasing compatibility, providing the practitioner with a natural reading order in which the most promising candidates are immediately accessible.

To support interpretability, the system additionally produces linguistic match reasons for each result derived from the component scores and rule-firing conditions of the inference engine.

These reasons are expressed in terms accessible to musically literate practitioners and communicate the specific basis for each recommendation, enabling users to understand why a particular song was ranked highly without requiring any knowledge of fuzzy logic implementation. A secondary defuzzification step maps the continuous match score to one of three affinity categories: scores above 0.7 are classified as High affinity, scores between 0.4 and 0.7 as Medium affinity, and scores below 0.4 as Low affinity. These categories are presented as color-coded badges in the Quick Match panel, providing an immediately accessible summary of match quality for users who do not wish to inspect component scores in detail. The combination of a continuous numerical ranking, linguistic explanations, and categorical affinity badges constitutes a complete defuzzification output designed to support efficient and informed creative decision-making.

IX. IMPLEMENTATION

MashHub is a client-side web application developed using React 19 and TypeScript, designed to support music library management and harmonic compatibility analysis for DJs, producers, and mashup creators. The system operates entirely within the browser environment and requires no server-side computation for its core matching functionality. Persistence is implemented through a dual-backend strategy in which Supabase, backed by a PostgreSQL relational database, serves as the primary cloud storage provider, while an IndexedDB store managed through the Dexie 4 library functions as the automatic local fallback when Supabase is unavailable or unreachable. A health-check mechanism executed at application startup determines which backend is active, and a higher-order function designated withFallback wraps

every service method to route operations accordingly without surfacing the distinction to the user interface layer.

The implementation is organized around three functional concerns. The first is user interaction, encompassing the React component tree through which users manage songs, organize projects, and invoke search and matching operations. The second is data processing, which includes the service abstractions responsible for reading, writing, and transforming data across both persistence backends. The third is intelligent scoring, which is realized through two independent fuzzy subsystems: a weighted approximate string matching engine built on Fuse.js 7 for text retrieval, and a custom Sugeno-type fuzzy inference system (FIS) for harmonic compatibility assessment based on BPM proximity, key distance, and section-level structural alignment. Together, these three concerns form a coherent pipeline in which user-specified criteria are translated into graded compatibility scores that rank candidate songs by their musical suitability for a given context.

A. User Interface

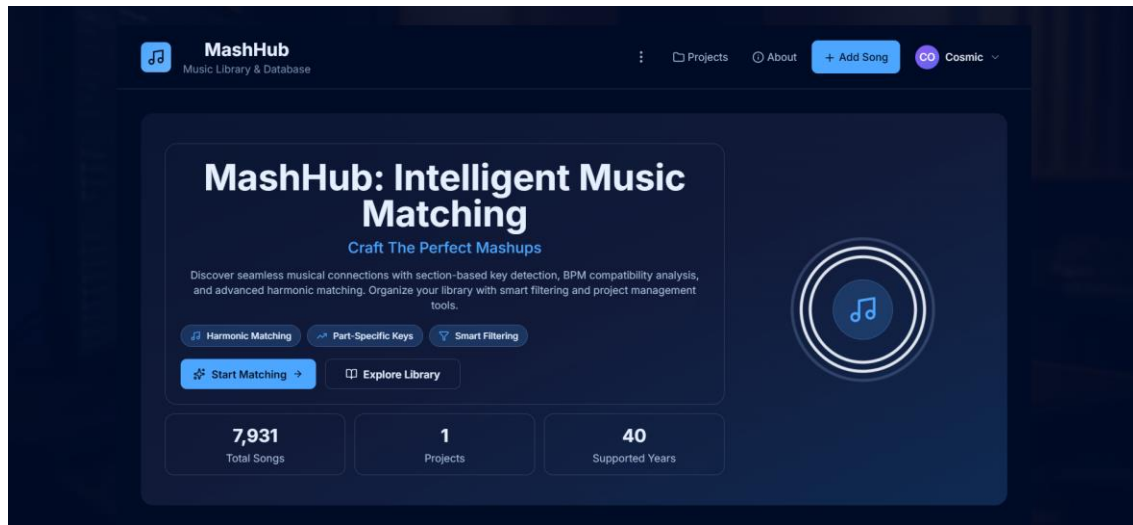


Figure 10.1. Home Page.

Styling is implemented with Tailwind CSS, which provides a utility-first class system that enables rapid layout construction while maintaining visual consistency across all breakpoints through responsive prefix modifiers. Animated transitions between interface states, including the expansion of section cards, the slide-in behavior of the SuggestionDrawer, and the collapse of filter blocks, are managed by Framer Motion, which integrates declarative animation primitives directly into the React component tree. Icon rendering throughout the interface is handled by Lucide React, a tree-shakeable icon library that contributes no unused assets to the production bundle. The drag-and-drop reordering functionality present in the Kanban board and Megamix timeline views is implemented using the @dnd-kit library, which provides pointer and touch event normalization, sortable context management, and accessible keyboard interaction support. The complete interface is delivered as a static asset bundle that requires no server-side

rendering, enabling deployment to any static hosting provider and supporting offline usage in conjunction with the IndexedDB persistence layer.

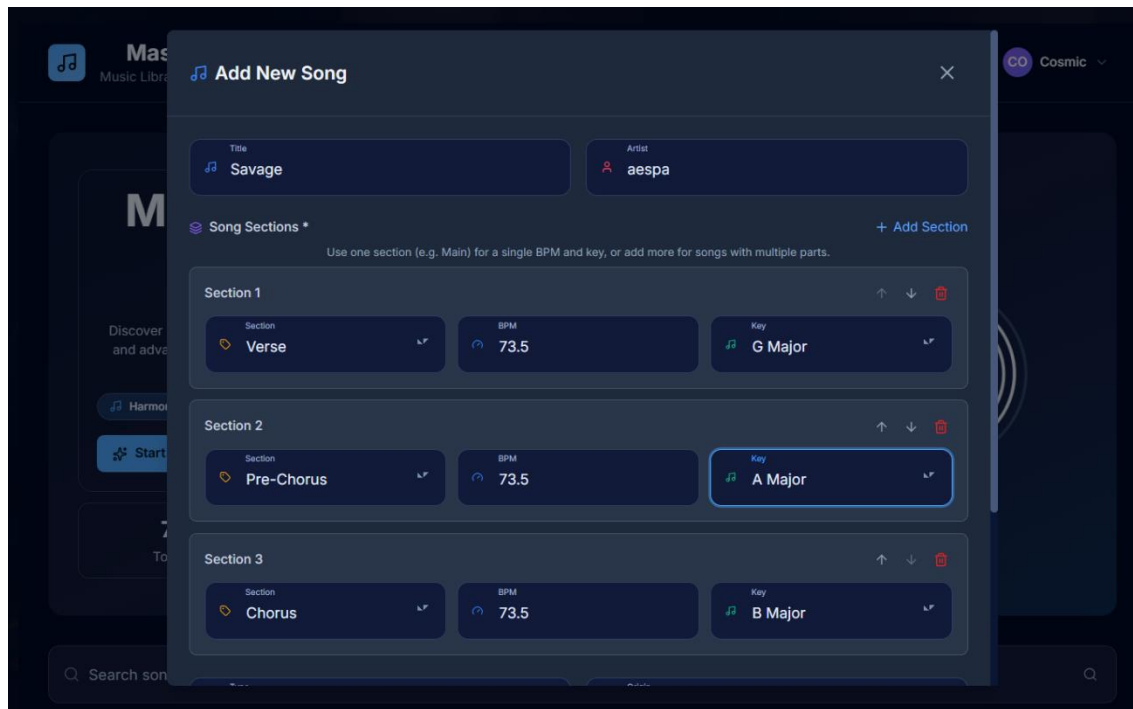


Figure 10.2. Adding a song.

The song management interface provides the primary mechanism through which the MashHub library is populated. Each song entity in the system is modeled as a compound structure comprising top-level bibliographic metadata and one or more associated section records. The top-level fields include the song title, artist name, type, origin, release year, season, and optional notes. Musical properties, specifically BPM and key, are not stored at the song level but are instead delegated to the section level, reflecting the system's recognition that a single song may exhibit distinct harmonic characteristics across different structural segments such as the Intro, Verse, Prechorus, Chorus, and Bridge.

When a user initiates the creation of a new song, a modal dialog is presented in which the top-level metadata fields are populated alongside a section editor. The section editor allows the user to define one or more named sections, each carrying a part label, a BPM value, a musical key, and a sequential order index. Upon submission, the song and its associated sections are persisted through the `songService` abstraction, which routes the write operation to Supabase when operating in cloud mode or to the Dexie IndexedDB store when operating in local mode. The song list on the main library page is updated immediately following a successful write, displaying all available songs with their primary BPM and primary key values computed from the first section in sequential order.



Figure 10.3. Search Subsystem.

The search subsystem is implemented through the `SearchService` class, which maintains an in-memory Fuse.js index built over the loaded song collection. Unlike exact-match retrieval, the Fuse.js engine applies the Levenshtein distance algorithm to compute approximate similarity

between the user's query string and the indexed field values across multiple metadata dimensions simultaneously. The index is configured with a similarity threshold of 0.6, a maximum character distance of 100, and a minimum matching character length of 2, with the `ignoreLocation` and `findAllMatches` parameters enabled to permit matches occurring at any position within a field value. Field weights are assigned as follows: the title field carries a weight of 0.40, the artist field 0.30, the type field 0.15, the origin field 0.10, and the part field 0.05, reflecting the relative importance of each field to song identification. The index is maintained incrementally using the `updateSongs`, `addSong`, and `removeSong` methods on the service's module-level Fuse singleton, so that library modifications are reflected in subsequent searches without requiring a full index reconstruction. Queries are submitted through the `AdvancedSearchBar` component with a debounce interval of 300 milliseconds, and results are returned as an ordered array of song records sorted by descending relevance score, optionally limited to a caller-specified maximum count.

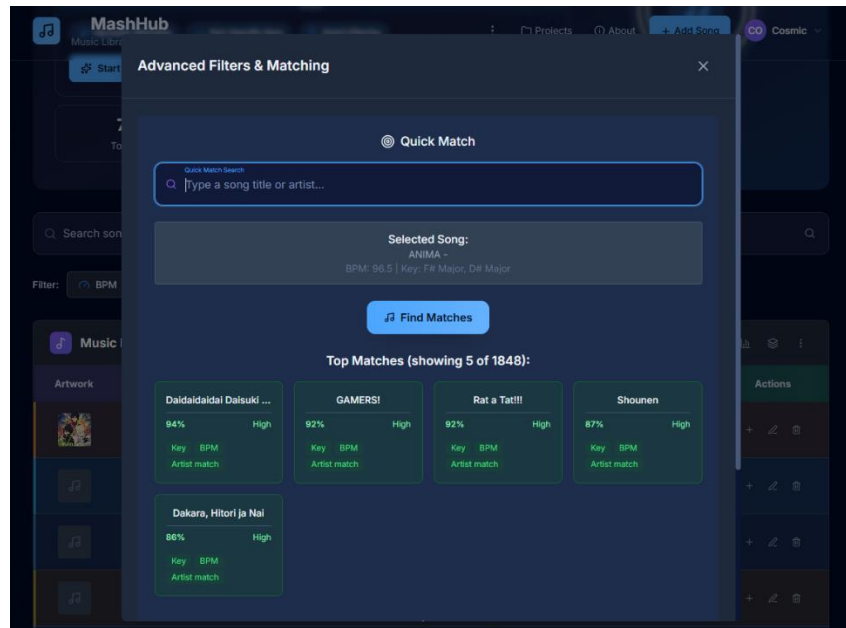


Figure 10.4. Quick Match.

The harmonic matching system is accessed through the `AdvancedFiltersDialog` component, which exposes two distinct interaction modes. In the standard filtering mode, the user specifies one or more harmonic criteria including a target BPM with an optional tolerance range or minimum-maximum bounds, a set of acceptable musical keys selected via checkbox controls, a year range, a vocal status category, and a text search term. These criteria are assembled into a `MatchCriteria` object and evaluated against the full song library by the `evaluateMatch` function in the matching service, which applies the weighted rule base of the standard Sugeno FIS to produce a ranked list of `MatchResult` objects. In the Quick Match mode, the user selects a reference song from a dropdown control within the filters dialog. The system immediately invokes the `getQuickMatches` function, which applies the section-focused weight distribution, and returns a ranked list of compatible songs annotated with affinity category labels. Songs scoring above 0.7 are classified as high affinity, those between 0.4 and 0.7 as medium

affinity, and those below 0.4 as low affinity. Each result is accompanied by a set of human-readable reason strings describing the specific harmonic relationships that contributed to the score, such as a statement that the Chorus BPM of a candidate song falls within a specified tolerance of the reference song's Chorus BPM, or that the Verse keys of two songs are separated by a single semitone.



Figure 10.5. Project and Section-Based Matching.

The project management system allows users to organize songs into named containers called projects, each of which is further subdivided into first-class project sections with configurable harmonic constraints. A project section carries fields for a target BPM, minimum and maximum BPM bounds, a target key, and a Camelot wheel range value, which the matching layer consults when computing compatibility scores within a project context. Projects are typed

as one of four categories: Seasonal, Year-End, Song Megamix, or Other. The project type governs the rendering mode of the project workspace, with Seasonal, Year-End, and Other projects displayed as a Kanban board in which each project section is rendered as a column card, and Song Megamix projects displayed as a horizontal Megamix timeline. Songs are assigned to project sections through the `addSongToSection` method of the `projectService`, which creates a `ProjectEntry` record linking the song identifier to the section identifier along with an order index and an optional notes field. Reordering of songs within a section is supported through drag-and-drop interaction implemented via the `@dnd-kit` library, and cross-section movement is handled by removing the entry from its source section and creating a new entry in the target section. The `SuggestionDrawer` component, accessible from the project workspace toolbar, invokes the matching service against the harmonic constraints of a selected project section and presents a ranked list of compatible songs from the library that have not yet been added to that section.

B. Fuzzy Logic

Fuzzification is the process by which crisp numerical and categorical inputs are transformed into graded membership values ranging continuously from zero to one, representing the degree to which an input satisfies a given fuzzy set. MashHub implements three distinct fuzzification procedures corresponding to BPM proximity, key compatibility, and section name normalization.

BPM fuzzification is implemented in the bpmMatching.ts utility module through two functions that apply different membership curve shapes depending on the operational mode. The standard matching function, `getBpmCompatibilityScore`, accepts an array of BPM values from a candidate song's sections and a target BPM derived from the active filter criteria. It identifies the section BPM in the candidate array that minimizes the absolute distance from the target and applies a linear decay formula to produce a membership value. If the minimum distance exceeds the caller-specified maximum delta, a membership of zero is returned immediately. Otherwise, the membership is computed as the maximum of zero and the quantity one minus the ratio of the best distance to the constant `BPM_SCORE_DENOMINATOR`, which is set to 22.5. This produces a trapezoidal-style membership function in which BPM differences of zero yield full membership, differences approaching 22.5 BPM yield near-zero membership, and differences beyond the specified delta are excluded entirely.

```
function getBpmCompatibilityScore(  
  candidateBpms: number[],  
  targetBpm: number,  
  maxDelta: number  
): number {  
  const bestMatch = Math.min(  
    ...candidateBpms.map(b => Math.abs(b - targetBpm))  
  );  
  if (bestMatch > maxDelta) return 0;  
  return Math.max(0, 1 - bestMatch / BPM_SCORE_DENOMINATOR);  
}
```

Figure 10.6. Function: `getBpmCompatibilityScore()`.

The Quick Match BPM function, `getQuickMatchBpmScore`, applies a piecewise curve that approximates a smooth bell-shaped profile when plotted symmetrically across the full range of positive and negative BPM differences, as illustrated in Figure [n+1]. This curve was calibrated to retain high membership values across the BPM differences most commonly encountered in live DJ transitions. Differences of zero BPM produce a membership of 1.0, differences of five BPM produce 0.9, and differences of ten BPM produce 0.8. Differences beyond ten BPM are handled by a second piecewise segment in which the membership decays from 0.7 at eleven BPM to zero at twenty BPM. The two segments together produce a curve that avoids an abrupt perceptual discontinuity at the ten-BPM boundary, ensuring that candidates in the vicinity of that threshold are scored in proportion to their actual harmonic proximity.

```
function getQuickMatchBpmScore(bpm1: number, bpm2: number):  
number {  
  const diff = Math.abs(bpm1 - bpm2);  
  if (diff <= 10) return 1 - diff * 0.02;  
  return Math.max(0, 0.7 - (diff - 11) * (0.7 / 9));  
}
```

Figure 10.7. Function: `getQuickMatchBpmScore()`.

Key fuzzification is implemented in the `keyNormalization.ts` utility module using the concept of circular semitone distance over the twelve-tone equal temperament pitch class space. The function `calculateKeyDistance` takes two key strings, resolves each to a pitch class index between zero and eleven, and computes the minimum circular distance between them using the precomputed `SEMITONE_DISTANCE_MAP` lookup table, a 12×12 matrix of pre-evaluated circular distances that eliminates repeated modular arithmetic at query time. A pitch-class distance of zero yields a membership of 1.0 regardless of whether the two keys share the same modality,

reflecting the design decision that parallel major and minor keys occupying the same tonal center are treated as fully compatible. For non-zero distances, the function applies a bell-shaped decay curve, illustrated in Figure 10.8, that produces the following reference membership values: one semitone maps to 0.95, two semitones to 0.90, three semitones to 0.80, and six semitones to 0.0. This bell-shaped profile provides meaningfully greater tolerance for harmonically adjacent keys than a linear decay formula would, reflecting the perceptual reality that keys separated by one or two semitones retain substantial tonal overlap in practice. At six semitones, corresponding to the tritone interval, the membership reaches zero, denoting the maximum harmonic distance within the chromatic scale beyond which no compatibility is credited.

```
function calculateKeyDistance(key1: string, key2: string): number {
  const { pitchClass: pc1 } = parseKey(key1);
  const { pitchClass: pc2 } = parseKey(key2);
  const distance = SEMITONE_DISTANCE_MAP[pc1][pc2];
  if (distance === 0) return 1.0;
  if (distance >= KEY_MAX_SEMITONE_DISTANCE) return 0.0;
  // Bell-shaped decay: reference values at d=1→0.95, d=2→0.90,
  // d=3→0.80, approaching 0.0 at d=6
  return computeKeyMembership(distance);
}

const getQuickMatchKeyScore = calculateKeyDistance;
```

Figure 10.8. Function: calculateKeyDistance().

The key score, derived from the bell-shaped decay function of `calculateKeyDistance`, rewards candidates across a continuous and smoothly graded spectrum of tonal proximity. An exact pitch-class match between two keys, whether in the same or parallel mode, yields a key membership of 1.0, reflecting the full harmonic compatibility of tonal centers that share the same root. At one semitone distance, the membership value of 0.95 indicates that keys separated by a

semitone retain very high compatibility, consistent with the common practice of mixing in adjacent keys on the Camelot wheel. At two semitones, a membership of 0.90 continues to reflect substantial tonal overlap, while at three semitones the value of 0.80 marks the boundary beyond which compatibility begins to decline more steeply. At six semitones, corresponding to the tritone, the membership reaches 0.0, denoting the maximum harmonic distance within the chromatic scale. The bell-shaped profile of this curve is consequential in practice: it awards meaningfully higher compatibility scores to songs in closely adjacent keys than a linear decay function would, increasing the likelihood that musically viable candidates appear in the upper tier of ranked results even when no exact key match exists in the library. This graduated representation of partial compatibility reduces false negatives in the recommendation output and provides practitioners with a broader and more musically realistic candidate set than hard-threshold key filtering could support.

Section name fuzzification is performed by the `normalizeSectionName` function in `sectionNormalization.ts`. This function maps diverse and user-generated section label variants present in the song database onto a canonical vocabulary of base section names. Labels such as "Verse A," "Verse B," and "Verse 2" are resolved to the canonical form "Verse," while labels such as "Intro Drop" are resolved to "Intro" and "Chorus 2" to "Chorus." By assigning full membership to a canonical base section, the system enables structural matching between songs whose sections carry differently specified labels, extending the harmonic inference process to accommodate the terminological variability inherent in real-world music metadata.

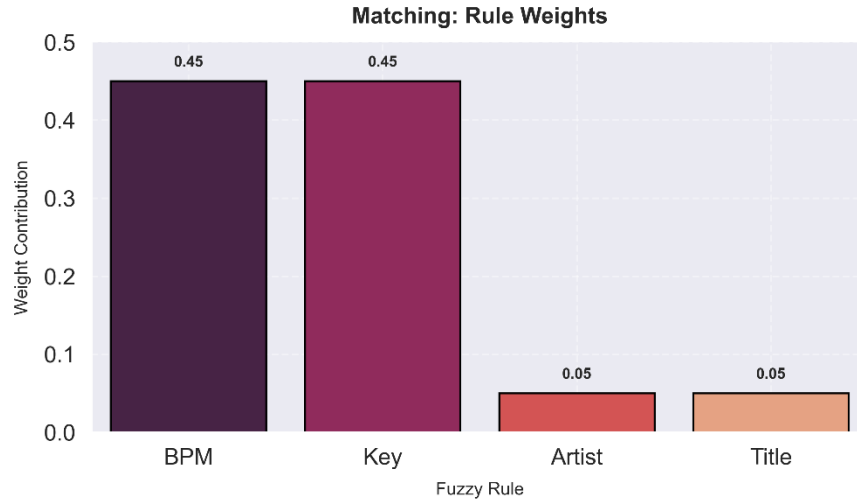


Figure 10.9. Matching: Rule Weights.

The rule base of the MashHub FIS is implemented as a collection of weighted IF-THEN rules whose consequents are singleton constants in the Sugeno formulation. All weight constants are defined centrally in `src/constants/index.ts` to facilitate tuning and reproducibility. The standard matching mode, implemented in the `evaluateMatch` function of `matchingService.ts`, applies four rules. The BPM compatibility rule contributes the product of the BPM membership value and the constant `MATCH_WEIGHT_BPM`, set to 0.45. The key compatibility rule contributes the product of the key similarity score and `MATCH_WEIGHT_KEY`, also set to 0.45. The artist correspondence rule contributes a flat bonus equal to `MATCH_WEIGHT_ARTIST`, set to 0.05, when the candidate song's artist field exactly matches the criterion artist. The title text containment rule contributes `MATCH_WEIGHT_TITLE`, set to 0.05, when the candidate's title contains the search string. The four weights sum to 1.0, meaning a candidate that fully satisfies all four conditions simultaneously achieve a maximum possible `matchScore` of 1.0.

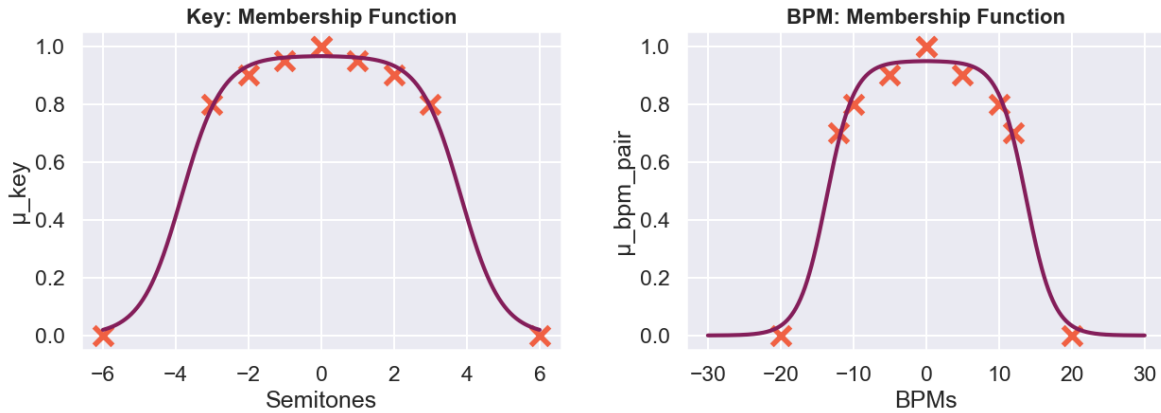


Figure 10.10. Key, BPM: Membership Functions.

The BPM and key membership functions illustrated in Figures [n+1] and [n+2] define the fuzzification curves that translate raw numeric differences into graded compatibility degrees. Both curves exhibit a bell-shaped profile centered at zero distance, reflecting the design objective that compatibility degrades smoothly and symmetrically as the musical properties of two sections diverge. The BPM curve reaches full membership of 1.0 at zero difference and retains high membership of 0.80 at a difference of ten beats per minute, a range that encompasses the natural tempo variation practitioners routinely bridge in live performance transitions. The key curve similarly reaches 1.0 at zero semitone distance and retains a membership of 0.90 at two semitones, acknowledging that keys separated by a whole step share substantial harmonic content and remain viable candidates for combination. At six semitones, corresponding to the tritone interval, both membership values reach 0.0, reflecting the musical convention that tritone-separated keys represent the maximum harmonic distance within the chromatic scale.

The discrete IF-THEN rules derived from these membership functions, together with their weighted consequents under the Sugeno formulation, are enumerated in Table 10.1 below.

Condition	Membership (μ)	Weighted Consequent (z)
IF section BPM difference is 0 BPM	$\mu_{\text{bpm_pair}} = 1.00$	$z_{\text{bpm_pair}} = 1.00 \times 0.45 = 0.450$
IF section BPM difference is 5 BPM	$\mu_{\text{bpm_pair}} = 0.90$	$z_{\text{bpm_pair}} = 0.90 \times 0.45 = 0.405$
IF section BPM difference is 10 BPM	$\mu_{\text{bpm_pair}} = 0.80$	$z_{\text{bpm_pair}} = 0.80 \times 0.45 = 0.360$
IF section BPM difference is 12 BPM	$\mu_{\text{bpm_pair}} = 0.70$	$z_{\text{bpm_pair}} = 0.70 \times 0.45 = 0.315$
IF section BPM difference is 20 BPM	$\mu_{\text{bpm_pair}} = 0.00$	$z_{\text{bpm_pair}} = 0.00 \times 0.45 = 0.000$
IF key distance is 0 semitones	$\mu_{\text{key}} = 1.00$	$z_{\text{key}} = 1.00 \times 0.45 = 0.4500$
IF key distance is 1 semitone	$\mu_{\text{key}} = 0.95$	$z_{\text{key}} = 0.95 \times 0.45 = 0.4275$
IF key distance is 2 semitones	$\mu_{\text{key}} = 0.90$	$z_{\text{key}} = 0.90 \times 0.45 = 0.4050$
IF key distance is 3 semitones	$\mu_{\text{key}} = 0.80$	$z_{\text{key}} = 0.80 \times 0.45 = 0.3600$
IF key distance is 6 semitones	$\mu_{\text{key}} = 0.00$	$z_{\text{key}} = 0.00 \times 0.45 = 0.0000$

Table 10.1. Fuzzy IF-THEN rules for BPM and key membership.

The table demonstrates the Sugeno inference structure directly: each row is a complete IF-THEN rule in which the antecedent specifies a crisp input distance, the membership column records the degree to which that input satisfies the fuzzy compatibility set, and the weighted consequent column records the contribution of that rule to the final matchScore. When multiple section pairs are evaluated during a Quick Match operation, the membership values across all applicable section pairings are computed individually and their mean is taken as the song-level

BPM or key compatibility score before multiplication by the respective weight constant, as described in the Inference Engine subsection below.

Section name fuzzification is performed by the `normalizeSectionName` function in `sectionNormalization.ts`. This function maps the diverse and user-generated section label variants present in the song database onto a canonical vocabulary of base section names. Labels such as "Verse A," "Verse B," and "Verse 2" are resolved to the canonical form "Verse," while labels such as "Intro Drop" are resolved to "Intro" and "Chorus 2" to "Chorus." By assigning full membership to a canonical base section, the system enables structural matching between songs whose sections carry differently specified labels, extending the harmonic inference process to accommodate the terminological variability inherent in real-world music metadata.

```
let matchScore = 0;
matchScore += bpmCompatibility * MATCH_WEIGHT_BPM; // 0.45
matchScore += keyScore * MATCH_WEIGHT_KEY;        // 0.45
if (criteria.artist && song.artist === criteria.artist)
  matchScore += MATCH_WEIGHT_ARTIST;                // 0.05
if (criteria.searchText && song.title.includes(criteria.searchText))
  matchScore += MATCH_WEIGHT_TITLE;                 // 0.05
```

Figure 10.11. Quick Match Mode.

The Quick Match mode applies a parallel four-rule set through the constants `QUICK_MATCH_WEIGHT_KEY` and `QUICK_MATCH_WEIGHT_BPM`, each set to 0.45, reflecting the primacy of section-level harmonic proximity in song-to-song compatibility assessment. Same-artist and same-origin bonuses are applied at `QUICK_MATCH_WEIGHT_ARTIST` and `QUICK_MATCH_WEIGHT_ORIGIN`, each set to 0.05. Before scoring, a type-consistency pre-filter

is applied: any candidate whose type field differs from that of the reference song is excluded from the result set entirely, functioning as a hard constraint rather than a soft weighted rule.

The inference engine aggregates the outputs of the rule base through a hierarchical weighted summation procedure that operates at two levels of granularity: the individual song section level and the song level. At the section level, the function `calculatePartSpecificKeyScore` accepts two songs and iterates over the sections of the reference song, identifying corresponding sections in the candidate song by comparing normalized part labels produced by `normalizeSectionName`. When one or more candidate sections with a matching normalized label are located, the function computes the key similarity between all pairwise combinations of keys from the reference section and the matched candidate sections using `getQuickMatchKeyScore`. The mean of all pairwise similarity values is then taken as the score for that target section. This mean-based aggregation treats all key pairings within a section as equally informative, avoiding the overestimation that would arise if only the single most favorable pairing were credited.

```

function calculatePartSpecificKeyScore(
  referenceSong: Song,
  candidateSong: Song
): number {
  const sectionScores: number[] = [];
  for (const refSection of referenceSong.sections) {
    const refPart = normalizeSectionName(refSection.part);
    const matches = candidateSong.sections.filter(
      s => normalizeSectionName(s.part) === refPart
    );
    if (matches.length === 0) continue;
    const keyPairs = refSection.keys.flatMap(rk =>
      matches.flatMap(m => m.keys.map(ck => getQuickMatchKeyScore(rk,
        ck)))
    );
    const sectionMean = keyPairs.reduce((sum, v) => sum + v, 0) /
    keyPairs.length;
    sectionScores.push(sectionMean);
  }
  if (sectionScores.length === 0) return fallbackKeyScore(referenceSong,
    candidateSong);
  return sectionScores.reduce((sum, s) => sum + s, 0) / sectionScores.length;
}

```

Figure 10.12. calculatePartSpecificKeyScore().

The per-section mean scores produced by this procedure are themselves averaged across all matched section pairs to yield a single part-specific key compatibility value in the range from zero to one. This two-stage aggregation, in which the mean similarity is computed across key pairs within each section and then the mean is again taken across sections, reflects a symmetric treatment of all harmonic evidence present in the data, preventing any single section or key pairing from exerting disproportionate influence on the overall score. Section-level BPM scoring in Quick Match mode follows the same mean pairwise aggregation pattern, computing `getQuickMatchBpmScore` for all BPM pairs across matched sections and averaging the results.

When a candidate song contains no section-level data, a full-song key fallback procedure compares the candidate's global key against each section key of the reference song and computes the mean of the resulting similarities, preserving the ability to score candidates regardless of the granularity of their metadata.

At the song level, the inference engine accumulates weighted rule outputs into a scalar `matchScore` through direct summation, consistent with the Sugeno model's use of a weighted average of singleton consequents.

Defuzzification in the MashHub FIS converts the aggregated scalar scores into an ordered ranking of candidate songs. Because the Sugeno model produces a crisp numerical output directly from the weighted aggregation procedure, no centroid computation or area-based integration is required. The defuzzification step is implemented as a descending sort of the `MatchResult` array by the `matchScore` field, placing the most compatible candidates at the top of the output list.

```
return results.sort((a, b) => b.matchScore - a.matchScore);
```

Figure 10.13. MatchResult object.

In addition to the numeric ranking, each `MatchResult` object is decorated with an array of human-readable reason strings generated during the inference stage. These strings provide a linguistic summary of the specific harmonic relationships that contributed to each score, such as identifying that a candidate song's Chorus BPM matches the reference within a tolerance of three beats per minute, or that the Verse keys of the two songs are separated by a single semitone and therefore receive a compatibility score of 0.9. This linguistic output constitutes a second form of

defuzzification in which the graded internal scoring is translated into actionable textual explanations accessible to non-expert users.

The end-to-end processing flow of MashHub begins when a user specifies matching criteria through either the standard filter interface or the Quick Match selector in the `AdvancedFiltersDialog` component. The component assembles the user's selections into a `MatchCriteria` object containing the target BPM, acceptable key set, vocal status, type, year range, search text, and optionally a reference song identifier for Quick Match mode. This criteria object is passed to the matching service, which first retrieves the complete song collection from the `songService`, including all associated section records for each song. The songs are then filtered by any hard-exclusion criteria such as vocal status or type category before the fuzzy scoring procedure is applied.

For each candidate song passing the initial filter, the matching service invokes the BPM fuzzification function to produce a BPM membership value, the key fuzzification function to produce a key similarity score, and the text and metadata comparison procedures to evaluate the remaining rule conditions. In Quick Match mode, the section-level inference procedure is additionally invoked to compute part-specific BPM and key scores using the reference song's section data. The weighted rule outputs are accumulated into a final `matchScore`, the result is stored as a `MatchResult` object carrying the score components and reason strings, and the complete result array is sorted in descending order of `matchScore` before being returned to the presentation layer. The `AdvancedFiltersDialog` renders the sorted results as a list of match cards,

each displaying the candidate song's title, artist, and primary metadata alongside its affinity label and the associated reason strings.

The `matchScore` produced by the MashHub fuzzy inference system represents a normalized composite measure of harmonic compatibility between a candidate song and a specified reference context, expressed as a value in the continuous interval from zero to one. Higher values indicate greater overall compatibility across the evaluated dimensions. In both standard and Quick Match modes, the four active rules have weights summing to 1.0, so a candidate that achieves perfect membership on every rule simultaneously receives a `matchScore` of 1.0. In practice, scores are distributed across the full range, with the affinity thresholds at 0.7 and 0.4 providing operationally useful categorical distinctions for the user interface.

The BPM and key scores make the largest individual contributions to the final `matchScore`, collectively accounting for 90 percent of the total weight in both standard and Quick Match modes, with each contributing 0.45. This symmetric distribution reflects an empirical judgment that tempo alignment and tonal compatibility are of approximately equal importance in determining the musical viability of a transition. The BPM score, derived from either the trapezoidal standard function or the piecewise Quick Match curve, rewards candidates not only for close tempo alignment but also, through the `areBpmsHarmonicallyRelated` fallback function, for relationships at standard harmonic ratios such as half-time and double-time within a configurable percentage tolerance. The key score, derived from the linear circular semitone decay of `calculateKeyDistance`, rewards candidates across a continuous spectrum of tonal proximity rather than applying a binary compatible or incompatible judgment. An exact pitch-

class match between two keys, whether in the same or parallel mode, yields a key membership of 1.0, acknowledging that parallel major and minor relationships share sufficient tonal material to support harmonic transitions without penalty.

The flexibility conferred by fuzzy logic over strict binary filtering is most evident in cases where a candidate song does not exactly satisfy a criterion but remains musically viable. A candidate whose Chorus BPM differs from the target by eight beats per minute would be excluded entirely by a rigid threshold filter, yet receives a Quick Match BPM membership of 0.84 under the piecewise sigmoid curve, allowing it to appear in the ranked results with a reason string quantifying the deviation. A candidate whose Verse key is two semitones from the reference receives a key membership of approximately 0.667, reflecting its status as a harmonically adjacent but non-identical match rather than treating the relationship as a failure. These graded representations reduce false negatives in the recommendation output and provide users with a broader, more musically realistic set of candidates than hard-threshold filtering could support.

The section-based comparison procedure adds a further dimension of interpretive value by attributing compatibility scores to specific structural segments of each song rather than to the song as a whole. Because different sections of a song frequently exhibit distinct BPM and key characteristics, a global single-value comparison would systematically underestimate compatibility between songs that align well in their transitionally critical sections while differing in others. The mean-over-mean aggregation strategy employed by the section-level inference engine ensures that all available harmonic evidence within each matched section is weighted equally, and that all matched sections contribute symmetrically to the overall score. The resulting

part-specific reason strings allow users to identify precisely which structural segments of a candidate song are most compatible with the reference, enabling informed decisions about the specific musical points at which a transition between the two songs would be most coherent.

X. CONCLUSION AND FUTURE ENHANCEMENTS

MashHub demonstrates that a transparent fuzzy inference approach can address the fundamental limitations of binary filtering for music compatibility assessment while remaining computationally efficient, architecturally maintainable, and practically useful for the creative workflows of disc jockeys and mashup creators. By encoding musical compatibility knowledge as explicit membership functions and weighted rules rather than learned parameters, the system produces recommendations that are both musically coherent and fully interpretable, enabling practitioners to inspect, evaluate, and act on the reasoning behind each suggestion without requiring familiarity with the underlying mathematical framework. The section-based data model and hierarchical aggregation strategy extend the fuzzy matching framework to operate at the structural level relevant to mashup planning, surfacing compatible song pairs that whole-song analysis would miss and directly addressing a limitation that no existing metadata-based tool has resolved.

The dual-layer architecture, combining Fuse.js-powered fuzzy text retrieval with a custom harmonic matching pipeline, demonstrates that fuzzy logic can be applied productively at multiple levels of a music library management system, addressing both the text retrieval problem of finding songs by approximate metadata and the compatibility scoring problem of ranking songs by harmonic and rhythmic proximity to a target. The dual-backend persistence model, with

Supabase as the primary cloud store and IndexedDB as the local fallback, provides the resilient data layer required for reliable operation in the varied network conditions of professional creative environments. Together, these architectural decisions constitute a coherent and practically effective approach to intelligent music library management that can serve as a reference design for future systems in this domain.

The limitations of the system are important to acknowledge for the guidance of future development. MashHub performs no audio signal processing; beats-per-minute and key values are stored as manually entered or imported metadata, and the accuracy of matching results is therefore contingent on the accuracy of the stored data. Errors or inconsistencies in the metadata propagate directly to matching results without detection. The system does not support real-time audio playback, beats-per-minute detection, or key detection from audio files, which means that the database cannot be automatically populated or verified from audio content. The matching algorithms were designed and validated against a library of anime, game, and Japanese popular music, and their calibration, particularly the beats-per-minute tolerance denominator and the key penalty values, reflects assumptions appropriate to that genre context that may require recalibration for other repertoires. Additionally, the matching parameters are fixed at deployment and cannot be personalized based on user feedback or usage patterns; the system does not implement any form of learning or adaptation.

Future enhancements can be organized around three principal directions. The first direction is adaptive rule weighting through expert feedback, in which practitioners rate the quality of recommendations over time and the system adjusts the weights of its fuzzy rules to

better reflect their personal compatibility preferences. This would transform the currently static rule base into a personalized model calibrated to individual creative judgment, potentially increasing recommendation relevance substantially for practitioners whose compatibility intuitions differ from the default calibration.

The second direction is the implementation of a guided song addition workflow to ensure the accuracy and integrity of the database. In this approach, all newly submitted song analyses from users are first stored in separate staging tables, specifically "new_songs" and "new_song_sections", rather than being immediately integrated into the main dataset. A designated role known as an Evaluator is introduced, consisting of users with sufficient expertise in analyzing song attributes. These evaluators are responsible for reviewing each submitted entry to determine its correctness. They are given the ability to approve, revise, or reject submissions. When a submission is approved, it is transferred into the primary tables "songs" and "song_sections". If revisions are necessary, the evaluator may modify the data before approval. If a submission is rejected, it is removed entirely from the staging tables. This controlled validation process ensures that only verified and high-quality data is incorporated into the system, thereby maintaining the reliability of the fuzzy logic computations and overall system outputs.

Lastly, the third direction is performance optimization, particularly as the dataset scales. The current in-memory computation approach is efficient for moderate-sized libraries, but as the number of songs and sections increases, the computational cost of pairwise comparisons may become significant. Introducing indexing strategies, caching of previously computed match

scores, or incremental recomputation techniques would help maintain responsiveness. This is especially relevant for real-time interactions such as Quick Match and Suggestion Drawer updates, where latency can negatively impact user experience.

REFERENCES

- Bello, J. P., & Pickens, J. (2005). A robust mid-level representation for harmonic content in music signals. In Proceedings of the 6th International Society for Music Information Retrieval Conference (ISMIR), 304-311.
- Brewster, B. and Broughton, F. (2006). *Last Night a DJ Saved My Life: The History of the Disc Jockey*. Updated ed. London: Headline.
- Brøvig-Hanssen, R. and Harkins, P. (2012). Contextual incongruity and musical meaning in mashups. *Popular Music*, 31(1), pp. 87–104.
- Butler, M.J. (2006). *Unlocking the Groove: Rhythm, Meter, and Musical Design in Electronic Dance Music*. Bloomington: Indiana University Press.
- Casey, M., Veltkamp, R., Goto, M., Leman, M., Rhodes, C., & Slaney, M. (2008). Content-based music information retrieval: Current directions and future challenges. *Proceedings of the IEEE*, 96(4), 668-696.
- Covach, J. (2005). Form in rock music: a primer. In: D. Stein, ed., *Engaging Music: Essays in Music Analysis*. Oxford: Oxford University Press, pp. 65–76.
- Dannenberg, R. B. (2018). Toward automated holistic beat tracking, music analysis, and understanding. In Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR), 687-693.

Dexie.js. (2023). Dexie.js: A minimalistic wrapper for IndexedDB. Retrieved from <https://dexie.org>

Distinct Mastering. (2024). Why Mixed in Key, Rekordbox, Serato, and Splice might be misleading you. <https://distinctmastering.com/post/why-mixed-in-key-rekordbox-serato-and-splice-might-be-misleading-you>

Fuse.js. (2023). Fuse.js: Lightweight fuzzy-search in JavaScript. Retrieved from <https://fusejs.io>

Gruhne, M., Dittmar, C., & Gaertner, D. (2009). Improving rhythmic similarity computation by beat histogram transformations. In Proceedings of the 10th International Society for Music Information Retrieval Conference (ISMIR), 177-182.

Gunkel, D.J. (2008). Rethinking the digital remix: mashups and the metaphysics of sound recording. *Popular Music and Society*, 31(4), pp. 489–510.

Henderson, M. [DJ Endo]. (2013). Dubspot lab report: Mixed In Key vs. Beatport key detection. Dubspot. <https://blog.dubspot.com/dubspot-lab-report-mixed-in-key-vs-beatport>

Korzeniowski, F., & Widmer, G. (2017). End-to-end musical key estimation using a convolutional neural network. In Proceedings of the 25th European Signal Processing Conference (EUSIPCO) (pp. 966–970). IEEE. <https://arxiv.org/abs/1706.02921>

Levitin, D.J. (2006). *This Is Your Brain on Music: The Science of a Human Obsession*. New York: Dutton.

- Mamdani, E. H., & Assilian, S. (1975). An experiment in linguistic synthesis with a fuzzy logic controller. *International Journal of Man-Machine Studies*, 7(1), 1-13.
- Parrser. (2024). The Spotify API collapse: What actually broke and why.
<https://parrser.com/spotify-collapse>
- Schreiber, H., & Muller, M. (2018). A single-step approach to musical meter estimation using a convolutional neural network. In *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR)*, 98-105.
- Takagi, T., & Sugeno, M. (1985). Fuzzy identification of systems and its applications to modeling and control. *IEEE Transactions on Systems, Man, and Cybernetics*, 15(1), 116-132.
- VirtualDJ Forums. (2019, March). Key detection issues [Forum thread]. Atomix Productions.
https://www.virtualdj.com/forums/228037/VirtualDJ_Technical_Support/Key_Detection_Issues.html
- VirtualDJ Forums. (2022, December). Use STEMS part to improve BPM and key analysis [Forum thread]. Atomix Productions.
https://www.virtualdj.com/forums/249870/Wishes_and_new_features/Use_STEMS_part_to_improve_BPM_and_Key_analysis.html
- Wearecrossfader. (2024). Mixed In Key 11: The ultimate key detection tool for DJs?
<https://wearecrossfader.co.uk/blog/mixed-in-key-11>

Wróblewski, P., Lisowski, M., & Ziółko, B. (2022). A comparison of the music key detection approaches utilizing key-profiles with a new method based on the signature of fifths. *Applied Sciences*, 12(21), 11261. <https://doi.org/10.3390/app122111261>

Zadeh, L.A. (1965). Fuzzy sets. *Information and Control*, 8(3), pp. 338–353.

Zak, A.J. (2001). *The Poetics of Rock: Cutting Tracks, Making Records*. Berkeley: University of California Press.